Cloud security guidance for developers

SnapVault project team | October 2026

Introduction

This guideline turns the eight Hack The Box Cloud Exploitation walkthroughs into reusable guidance for cloud developers. The aim is to stop an early weakness from turning into access to sensitive data or control of the host. A cloud provider will not stop an app from exposing credentials, over-granting permissions or running uploaded files, so build protection into each layer and confirm the app still works.

The guidance is seven controls, C1 to C7. Each keeps the same ID everywhere, so a walkthrough, a code review, a ticket or a test record can point to it, for example "this build fails C4". The walkthroughs use these same IDs to show how the hardened SnapVault build would have stopped each machine.

The main risks

The same weaknesses come up across the track. Secrets end up in client code, in old object versions, in repo history or in readable files. Identities and tokens carry more access than the workload needs. User input gets run, as an uploaded file or as a template rendered on the server. Storage a user can write to is wired to a web server that runs what it finds. Containers and pods run as root with the host or a privileged token in reach. A private network does not remove these risks when a compromised workload can still reach sensitive services.

The seven controls

ID	Control	What it stops
C1	No secrets in the web page	Credentials shipped to the browser for any visitor to read.
C2	A least-privilege app key	A leaked app key being used for far more than the app needs.
C3	No versioning or stored secret	Old copies of secrets surviving in object versions after the live file looks clean.
C4	An upload allow-list with random names	Attacker-chosen file types and filenames landing in storage.
C5	Files streamed and never executed	Uploaded or stored files being run by the web server as code.
C6	A non-root locked-down container	A compromised workload becoming control of the host or cluster.
C7	Secrets kept out of version control	Secrets leaking through exposed repos, old commits or plaintext files.

Each control came from specific machines. Use this to move between the guideline and the evidence.

ID	Demonstrated by
C1	Gobox leaks a credential through client-side code, the same class as a key left in app.js.
C2	Gobox, Monteverde and Unobtainium turn one over-scoped identity or token into access far beyond the app.

ID	Demonstrated by
C3	Epsilon recovers live keys from an old artifact, the pattern behind a config.bak in an old object version.
C4	Bucket turns a writable bucket into a foothold by copying in a file the server later runs.
C5	Bucket and Gobox both pair a writable bucket with a site that executes PHP served from it.
C6	SteamCloud and Unobtainium escape through a root pod with a mounted token. Bucket privesc's through a root-run service.
C7	Vessel and Worker leak secrets from an exposed repo and its history. Monteverde leaks one from a cleartext file on a share.

Control guidance

Each control has a requirement, the practices that meet it, and a check. Record every check against the control ID.

C1: No secrets in the web page

Requirement. Browser code, page source and anything a browser can download hold no passwords, access keys or signing secrets.

- Keep secrets in a secrets manager or protected runtime config, and read them only on the server.
- A key that reaches the browser can be read by any visitor, so treat it as leaked and rotate it.

Check. Read the page source and bundled scripts. Confirm no credential shows up.

C2: A least-privilege app key

Requirement. The key the app uses can do only what it needs, on only the resources it needs.

- Give the app, admins and deployment jobs separate identities, and never share one key across them.
- Allow only the actions needed, for example read, write and list on one bucket and nothing else.
- Prefer short-lived credentials over long-lived keys stored in the container.

Check. Confirm the app identity cannot reach other buckets, admin functions or privileged secrets.

C3: No versioning or stored secret

Requirement. No secret sits in storage, and object versioning leaves no old copy of one to recover.

- Keep secrets out of buckets. A config or backup that holds one does not belong in object storage.
- Deleting the current object leaves earlier versions, so turn versioning off on buckets that could hold a secret, or scope the read key so it cannot fetch versions.
- Rotate any secret that was ever written to storage. Do not rely on deleting the file.

Check. Confirm old credentials fail and that listing or fetching old object versions returns no working secret.

C4: An upload allow-list with random names

Requirement. The server accepts an upload only when it matches an allow-list, and the server names the stored file.

- Check the extension, the detected content type and the size, and reject anything off the list.
- Generate a random server-side filename and never store the submitted one.

- Authorize uploads and downloads so a user cannot touch another user's objects.

Check. Upload test files with a bad type, an oversized body and shell characters in the name. Confirm each is rejected or renamed.

C5: Files streamed and never executed

Requirement. Stored files reach users as inert data and are never run by the server.

- Store uploads outside the executable web root, apart from application code.
- Serve a file by streaming it through the app after an auth check, not from a path the web server can run.
- A storage write must never become a PHP or ASPX deployment. The object store does not run PHP. The web server running files from writable storage does.

Check. Upload a script through the normal path and confirm it downloads as text and never executes.

C6: A non-root locked-down container

Requirement. The app runs in a container as a non-root user, with host access and extra privileges removed.

- Run as non-root, drop all capabilities, and set no-new-privileges.
- Use a read-only root filesystem with only the few writable paths the app needs.
- Do not mount the Docker socket or host directories. On Kubernetes, turn off anonymous Kubelet access and unused token mounts, and block hostPath and hostNetwork pods.

Check. Confirm the container runs as the intended non-root user and cannot reach the host, other workloads or a usable orchestrator token.

C7: Secrets kept out of version control

Requirement. No secret is committed to a repository, and none survives in its history.

- Keep secrets in an untracked .env file, and commit only a .env.example template with placeholders.
- A secret deleted in a later commit still sits in history, so treat any committed secret as leaked and rotate it.
- Block .git at the web server and deploy build artifacts, not the repo. Store the admin password as a hash, never plaintext.

Check. Confirm no secret appears in the current tree, in history or in a readable file or share, and that .git is not reachable over the web.

Supporting practices

These back the seven controls but do not each need an ID.

- **Treat user input as data.** Validate request types and allowed fields on the server, bind database parameters, and never render user input as a template. Keep authorization flags server-side, reject keys like __proto__, and pass arguments as an array rather than building a shell command from user input.
- **Protect the pipeline.** Require authenticated repo access, protect production branches with review, and keep build and deploy out of the web root so a commit does not flow into a live site. Run build agents under low-privilege identities, and control who can queue builds or administer pipelines.
- **Patch components.** Update dependencies, images, web-server modules and helpers, and confirm the fix is in the deployed build. An outdated component carries known chains, and a SUID helper on an unpatched version is privilege-escalation surface.
- **Monitor and respond.** Collect auth, storage, app and pipeline logs without recording secrets. Alert on repeated failed logins, odd downloads and privileged changes. Keep tested backups, and document how to isolate a compromised workload and revoke its credentials.

Checking the protections

Test each control on its own before deployment. An attack blocked at the first step does not prove the later ones work. Use harmless test files and controlled accounts, then record the expected result, the actual result and the evidence against the control ID.

Give every open finding an owner and retest after changes. Developers own the app controls. Cloud and DevOps engineers own the infrastructure and deployment. Security reviewers sign off before the team calls the environment hardened.